SMTFL: Secure Model Training to Untrusted Participants in Federated Learning

Zhihui Zhao, *Member, IEEE*, Xiaorong Dong, Yimo Ren, Jianhua Wang, *Member, IEEE*, Dan Yu, *Member, IEEE*, Hongsong Zhu, *Member, IEEE*, and Yongle Chen, *Member, IEEE*

***Abstract*—Federated learning is an essential distributed model training technique. However, threats such as gradient inversion attacks and poisoning attacks pose significant risks to both privacy of training data and model correctness. We propose SMTFL, a novel approach for secure model training in federated learning. To safeguard gradients privacy against gradient inversion attacks, clients are dynamically grouped, allowing one client's gradient to be divided to obfuscate the gradients of other clients within the group. This method incorporates checks and balances to reduce the collusion for inferring specific client data. To detect poisoning attacks from malicious clients, we assess the impact of aggregated gradients on the global model's performance, enabling effective identification and exclusion of malicious clients. Each client's gradients are encrypted and stored, with decryption collectively managed by all clients. The detected poisoning gradients are invalidated from the global model through an unlearning method. Compared to related work, SMTFL does not rely on trusted participants, avoids the performance degradation caused with traditional noise-injection, and avoids complex homomorphic encryption during gradient aggregation. SMTFL is evaluated on five datasets, and these results demonstrate its effectiveness in defending against gradient inversion and poisoning attacks. The model accuracy is nearly restored to its pre-attack state when SMTFL is deployed. Furthermore, SMTFL achieves over 95% accuracy in identifying malicious clients while maintaining a false positive rate for honest clients within 5% , which is 6% lower than the latest methods.**

***Index Terms*—Federated learning, privacy protection, poisoning attacks, untrusted participants.**

Received 28 March 2025; revised 8 January 2026; accepted 10 February 2026. Date of publication 16 February 2026; date of current version 5 June 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62472302, in part by the Key Research and Development Program of Shanxi Province under Grant 202402150101007, and in part by the Open Project of the State Key Laboratory of Internet Architecture under Grant HLW2025MS31. Recommended for acceptance by Y. Deng. *(Corresponding author: Yongle Chen.)*

Zhihui Zhao is with the College of Computer Science and Technology, Taiyuan University of Technology, Taiyuan 030024, China, and also with the State Key Laboratory of Internet Architecture, Tsinghua University, Beijing 100084, China (e-mail: zhaozhihui@tyut.edu.cn).

Xiaorong Dong, Jianhua Wang, and Dan Yu are with the College of Computer Science and Technology, Taiyuan University of Technology, Taiyuan 030024, China, and also with the Shanxi Key Laboratory of Industrial Internet Security, Taiyuan 030024, China (e-mail: dongxiaorong0607@163.com; wangjianhua02@tyut.edu.cn; yudan@tyut.edu.cn).

Yimo Ren and Hongsong Zhu are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China, and also with the School of Cyberspace Security, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: renyimo@iie.ac.cn; zhuhongsong@iie.ac.cn).

Yongle Chen is with the College of Artificial Intelligence, Taiyuan University of Technology, Taiyuan 030024, China, and also with the Shanxi Key Laboratory of Industrial Internet Security, Taiyuan 030024, China (e-mail: chenyongle@tyut.edu.cn).

Digital Object Identifier 10.1109/TMC.2026.3664822

I. Introduction

FEDERATED learning (FL) [1] enables multiple clients to collaboratively train a model with the assistance of a central aggregation server. Instead of sharing raw data, each client uploads locally computed gradient updates. Owing to its data locality, scalability, and decentralized nature, FL is inherently well-suited to mobile and edge computing environments, where large numbers of mobile and Internet of Things (IoT) devices jointly train models under resource constraints while keeping data on-device. However, data remains a central challenge: it is valuable yet fragmented and highly heterogeneous across devices. With the growing demand for data driven by Large Language Models, Artificial Intelligence, and Edge Intelligence, FL increasingly relies on diverse participants—including outsourced third-party contributors [2], where users lacking sufficient data outsource their tasks and external parties provide gradient updates for rewards. These characteristics of mobile FL make the setting both powerful and inherently vulnerable.

Motivation: Privacy protection of training data and model correctness are critical for the reliable functioning of FL systems. As shown in Fig. 1, FL participants (i.e., clients and server) are untrusted, some may deviate from the established service rules: (1) *Gradient inversion attacks* [3]. Server and clients may be curious about others' training data, they can potentially recover the specific client's training data based on its gradient updates. (2) *Poisoning attacks* [4]. Malicious clients upload incorrect gradients, such as inverted gradients, to the server. This action causes the aggregated gradient to deviate from the benign gradient, thereby degrading the performance of FL model.

Some research has been addressing these security issues for FL. On the one hand, some work [3], [5] assumes that FL participants are semi-honest, i.e., participants perform their tasks correctly, but they attempt to violate the privacy rules and derive clients' training data. Secure aggregation techniques (homomorphic encryption [6], differential privacy (DP) [7], gradient masking [8], etc.) are widely used to protect data privacy. However, these methods may face challenges, including high computational overhead due to complex cryptographic operations, model performance degradation from noisy data insertion, and a focus on the transmission security of gradients rather than its computational security. On the other hand, some

1536-1233 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.

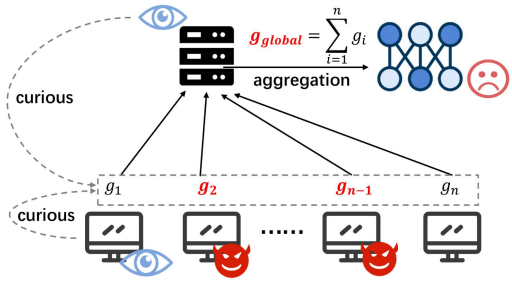


Fig. 1. The focused FL security issues in this paper.

studies (FLTrust [9], Ozdayi et al. [10]) have attempted to defend against poisoning attacks from malicious clients. They assume that the server is trustworthy and determine whether a client is malicious by collecting and training on a small amount of publicly available data pertaining to that client. According to the majority principle, the gradient direction uploaded by the majority of clients is deemed to be the correct direction. However, the server is semi-honest and can infer the clients' training data via these public data. It may misjudge poisoning attacks due to data bias and the scarcity of public data. Malicious clients may also collude [11], [12] to jointly upload gradients with the same but incorrect direction, or there may be multiple collusion groups, making it possible that there is no consensus on the gradient direction among clients. Eiffel [13] addresses both input privacy and integrity protection, but suffers from high complexity, making it challenging to balance model robustness with computational efficiency. Therefore, there are several *challenges* to address both security issues and limitations of existing work.

- During the interaction and gradient aggregation phase, both clients and the server may obtain a specific client's gradient updates if these gradients are aggregated in an unencrypted state and without added noise for obfuscation. This situation provides malicious participants with the opportunity to infer the training data of clients.
- In the absence of a trusted participant and without the need for the server to collect public data for each client, it becomes challenging to directly ascertain the correctness of a client's gradient. A single gradient update from a client with a limited data volume is likely to have a minimal impact on the global model's performance. The availability of the majority principle may be compromised by collusion attacks.
- If incorrect gradients from malicious clients are not detected promptly, they will be integrated into the global model during the undetected window period. Therefore, this scenario will degrade the long-term performance of the global model.

Methods: We propose a novel approach, especially **S**ecure **M**odel **T**raining to untrusted participants in **F**ederated **L**earning (**SMTFL**), to cope with the above challenges:

- To protect the gradients privacy, we group every three clients, where one client's gradient is dynamically divided into two shares to obfuscate the gradients of the other clients. The gradients from this group are ultimately aggregated by one client and uploaded to the server for getting the global model. To prevent some clients from obtaining a specific client's gradient through collusion attacks, our gradient aggregation rules establish a system of checks and balances among the group clients, such that colluders, in the act of exposing others' gradients, will also reveal their own.
- To detect poisoning attacks from malicious clients, we evaluate the impact of the aggregated gradients from a group of clients on the global model's performance. We assess this group of clients collectively based on the observed changes in the global model's performance. Although this rule may incidentally affect some honest clients during a single update, the dynamic grouping rule mitigates the overall impact on honest clients across each update session. When a client receives negative evaluations beyond a preset threshold, it will be considered as malicious and excluded from the FL system.
- To mitigate the impact of the aggregated poisoning gradients on the global model, each client's gradients are encrypted and regularly stored on a storage server. The decryption keys for these gradients are not held by any single entity but are collectively managed by all clients. Upon the detection of a malicious client, its gradients are decrypted under the consensus of the majority of clients. The server then uses the idea of federated unlearning [14] to mitigate the influence of the poisoning gradients.

Contribution: Our contributions are outlined as follows.

- SMTFL is a practical and secure model training scheme in FL scenarios without trusted participants. It can preserve gradient privacy against the gradient inversion attacks by servers and clients, and ensures the model performance, safeguarding it from poisoning attacks orchestrated by clients.
- In privacy protection, SMTFL does not use the noise (e.g., differential privacy) to avoid adverse impact on model performance. It eliminates the need for complex cryptographic operations in aggregation. To defense against poisoning attacks, FL system does not require trusted participants, and the server is not obligated to collect public data for each client beforehand. SMTFL can locate and remove malicious clients and mitigate the influence of aggregated poisoning gradients on the global model.
- SMTFL is evaluated on five datasets, and the results are encouraging. It effectively mitigates both poisoning and gradient inversion attacks, restoring model accuracy to levels close to the pre-attack state. It achieves over 95% accuracy in detecting malicious clients while keeping a false positive rate below 5% for honest clients. Across diverse data distributions and varying proportions of malicious clients, SMTFL maintains good performance. Comparisons with baseline and state-of-the-art methods demonstrate its advantages in global model accuracy, false positive rate, and other key metrics.

In the following, Section II introduces the related work. Sections III and IV describe the preliminaries, threat model, and

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON AMONG SMTFL AND RELATED WORK

Work	Poisoning attack	Gradient privacy	Malicious clients location	System robustness	Clean data in server	Limitation
FLTrust [9]	✓	×	✓	×	✓	• Server is trusted; • Server needs to collect and train on a small amount of publicly available data.
FLAIRS [15]	×	✓	✓	✓	×	• It depends on the trusted execution environment; • It does not support the non-iid data.
Naseri et al. [16]	×	✓	×	×	×	• It only focuses on the gradient's privacy; • It degrades the model's performance.
EIFFeL [13]	✓	✓	×	×	×	• Communication and computation costs are high, may not be available in real applications. • It is limited by the number of malicious clients.
FreqFed [17]	✓	×	✓	×	×	• It only focus on the poisoning attacks; • It is applicable to image data.
DDFed [18]	✓	✓	✓	✓	×	• Privacy is protected using homomorphic encryption; • The aggregation based on the majority principle may be compromised by collusion attacks.
SMTFL	✓	✓	✓	✓	×	—

security goals, respectively. The details of SMTFL are presented in Section V. Sections VI and VII provide the security analysis and experiment evaluation, respectively. Finally, Section VIII discusses and concludes this paper.

II. RELATED WORK

We review existing research on poisoning and gradient inversion attacks, along with current defense strategies, categorizing their approaches and discussing their limitations. A qualitative comparison is presented in Table I.

Anomaly detection-based methods: They defend against poisoning attacks by identifying and excluding the gradient updates of malicious clients. However, they are often limited by filtering strategies and data distribution assumptions. In term of filtering strategies, the similarity of gradients from different clients can be leveraged to filter a small number of malicious clients, but it is susceptible to collusion attacks. Fung et al. [19] employ the KMeans method to achieve filtering, but this method has high computational complexity, and malicious clients can circumvent defenses by submitting multiple backdoored samples. DDFed [18] uses homomorphic encryption for privacy protection, while aggregation based on the majority principle may be compromised by collusion attacks. Andreina et al. [20] and Cao et al. [9] assume that the server has access to clients' training data, which violates client's data privacy. Variations in data distribution (such as independent and identically distributed (iid) or non-iid) can cause differences in the clients' gradients. FLAIRS [15] and FreqFed [17] identify the abnormal gradients of malicious clients through anomaly detection algorithms (e.g., cosine distance and HDBSCAN), where these gradients are assumed to contain obvious outliers.

Differential privacy-based methods: Noise is added to the clients' gradients to prevent attackers from obtaining precise gradients, thereby avoiding the data reconstruction and mitigating negative impact on the performance of global model. Compared to anomaly detection methods, DP considers the risk of data leakage during gradient transmission. However, since the server cannot access clients' training data, it cannot accurately analyze and eliminate the effects of noise in gradient aggregation, which degrades the performance of global model. Naseri et al. [16] implement a collaboration between local and central DP, where clients add the noise to gradients while the server uses DP aggregation algorithms for gradient aggregation. As the number of malicious clients increases, it is more challenging for the server to analyze the noise. Based on the parameter clipping and Gaussian noise adding, McMahan et al. [21] modify the gradients to limit gradient sizes and protect data privacy, which has expensive computational costs. FLAME [22] combines the filtering of outlier detection, model clipping, and noise addition, but its privacy guarantees are only applicable to semi-honest server that adheres to the Secure Multi-Party Computation (SMPC) protocol.

Secure Aggregation-Based Methods: The goal is to achieve that the performance of global model is not significantly affected, even if some clients upload incorrect gradients. The server will not detect malicious clients during gradient aggregation. Instead, it employs specific aggregation strategies to enhance the robustness of global model. Multi-Krum method [23] repeatedly removes these gradient updates that are far from the geometric median of all updates. Trimmed Mean method [24] removes the maximum / minimum updates and computes the average of the remaining values. Although these methods mitigate the impact of incorrect gradients to some extent, the performance of global model still significantly declines as the number of attackers increases. Pasquini et al. [25] claim that current secure aggregation-based FL achieves a "false sense of security", i.e., it merely addresses superficial privacy protection without defending against client-side attacks. To achieve privacy protection and model correctness, Chowdhury et al. [13] split and aggregate the clients' gradients through the Shamir threshold secret sharing scheme and non-interactive proofs. This method has high computational complexity, making it challenging to balance the model robustness with the computational efficiency.

Overall, existing methods mitigate gradient inversion attacks and poisoning attacks, but they exhibit the limitations in complex

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.

attack scenarios and rarely address both security issues simultaneously. Against this background, the SMTFL offers a novel solution. Our proposed method not only effectively counters both gradient inversion attacks and poisoning attacks but also demonstrates outstanding robustness and stability in complex attack scenarios. Our method enables secure model training in FL involving untrusted participants, providing new perspectives and insights for addressing similar challenges.

III. Preliminaries

A. Primary Reasons That Clients are Untrusted

Clients are untrusted in the FL system. On one hand, clients are vulnerable to external attacks. Any computing-capable device may join the FL system, but individual users may lack the expertise to configure proper security strategies for their smart devices [26], [27]. Attackers can leverage these compromised clients to reveal others' gradients information or upload incorrect gradient updates. On the other hand, client's administrators may subjectively deviate from established rules. Semi-honest and malicious clients seek to obtain other clients' private training data. Malicious clients also upload incorrect gradient updates to sabotage the FL system (e.g., malicious competition). In some FL applications related to crowdsourced computing [2], some clients, facing data scarcity, may fabricate false gradients or manipulate gradients trained on public datasets prior to uploading them to the server. Additionally, some profit-driven clients may refrain from training, choosing instead to make minor modifications to the global model received from the server to conserve computational resources.

B. Federated Unlearning

Federated Unlearning (FUL) [14] focuses on the removal of specific clients' contributions or data samples from the global model, aiming to safeguard privacy and enhance security. The core lies in adjusting model parameters to achieve an effect equivalent to that of training without specific data, while avoiding complete retraining. Based on FUL, the server can, at user request, make the model forget the contributions trained by specific data. Therefore, server can enhance the model's security by making the model forget updates trained from incorrect or low-quality data. FUL includes contribution deletion [28], and training update correction [29], etc.

C. Threshold Encryption

Threshold encryption, a cryptographic technique, enhances the security and reliability of key management. This technique involves splitting a key into multiple shares, with the stipulation that the key can only be recovered when at least t shares are combined, thereby enabling data decryption. The method improves the security and fault tolerance of the key by eliminating single points of failure. Threshold encryption is primarily based on the Threshold Secret Sharing Protocol, such as the Shamir scheme [30].

Specifically, a key S is divided into N key shares $\{S_1, S_2, \ldots, S_N\}$, and a threshold value t is set to indicate that at least t key share unions are required to recover the original key S. For instance, consider the key S as a split secret, the Shamir scheme generates a random polynomial:

$$f(x) = S + a_1 x + a_2 x^2 + \cdots + a_{t-1} x^{t-1} \pmod{p},$$

where S is the secret, $a_1, a_2, \ldots, a_{t-1}$ are the randomly generated coefficients, p is the prime, and x is the index of the participant (e.g. $x_1, x_2, \ldots, x_N$). Each participant computes its key share $(x_i, f(x_i))$.

When at least t participants collaborate, given t key shares (x_1, y_1), $(x_2, y_2), \ldots$, (x_t, y_t), the secret S can be recovered using Lagrange interpolation:

$$S = \sum_{i=1}^{t} y_i \prod_{1 \le j \le t, j \ne i} \frac{x_j}{x_j - x_i} \pmod{p}.$$

IV. Threat Model and Security Goals

In the traditional threat model, participants are usually classified into 1) Honest: participants perform their assigned tasks correctly according to the established rules and do not violate the system's security rules. 2) Semi-honest: participants honestly follow the rules without cheating and other misbehavior, but they attempt to violate the data privacy rules. 3) Malicious: participants intentionally violate the security rules to fail the services and obtain privacy data. In the FL system consisting of a server and multiple clients, we consider a comprehensive threat scenario. We address not only poisoning attacks where clients disrupt model training but also gradient inversion attacks where both clients and the server may attempt to recover training data from others. To provide a formal security analysis, we define the *threat model and assumptions* as follows:

- *Server as semi-honest:* It correctly aggregates the uploaded gradients and returns the aggregated gradient to participating clients in each epoch. However, it is curious about clients' private training data and may attempt gradient inversion attacks. This assumption is reasonable: the server, operated by system administrators, is responsible for coordinating global model training with numerous clients. If the server has no concern for the quality of trained model (i.e., whether it is subject to poisoning attacks), it could potentially act as a poisoner itself, undermining the entire training process of the FL system.
- *Clients as semi-honest or malicious:* Semi-honest clients correctly compute and upload local gradients based on their private data, but may still attempt to infer other clients' data through unauthorized means (e.g., gradient inversion attacks). Malicious clients not only seek to get others' training data but also actively degrade model performance through poisoning attacks. They submit incorrect gradients in every epoch, i.e., their malicious behavior occurs at a 100% rate.
- *Collusion constraints and rationality assumption:* We assume that within a dynamically formed group of size $group_n$, the adversary cannot simultaneously control the server and all other $group_n - 1$ clients. This constraint is supported by the rationality assumption: we assume that all

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.

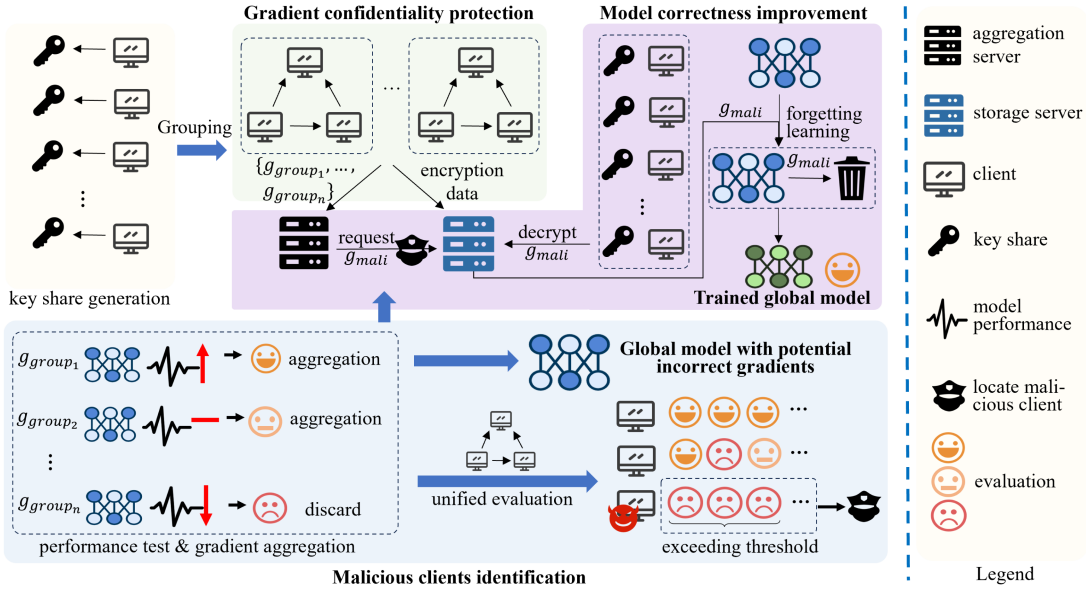


Fig. 2. The framework of SMTFL.

participating clients are rational and will not collude with other participants without sufficient incentive. In particular, since clients aim to obtain rewards such as access to other participants' training data or model benefits, we do not consider adversarial behaviors that involve sacrificing personal utility or yield no gain. Therefore, behaviors such as compromising one's own gradient privacy to infer others' data, or engaging in collusion without specific attack utility, are not considered in our threat model.

Most studies [3], [5] typically assume that clients and server are semi-honest, or that only clients are malicious. To our best knowledge, few works have concurrently addressed both scenarios. This paper considers a much stronger threat model and should meet the *security goals*: (1) Both clients and server are unable to get the gradients of specific clients; (2) Detecting and locating malicious clients that compromise FL system; (3) The performance of global model does not significantly deteriorate in the presence of malicious clients.

Formalization: Without loss of generality, we consider a FL system comprising clients $\{c_1, \ldots, c_m\}$ and an aggregation server s_a. During the k-th epoch, $\forall\, c_i$ utilizes its data d_{c_i} to train and obtain the updated local gradient g_i^k, which is uploaded to the s_a. s_a aggregate all g_i^k $(i \in [1, m])$ to derive the global gradient g^k, which is distributed to all clients for the $(k+1)$-th epoch. Every c_i and s_a attempt to access the g_j^k of c_j to infer the data d_{c_j} $(j \in [1, m])$. Furthermore, $\exists$ malicious c_i compromises the global model through uploading incorrect local gradients (e.g., inverting gradient direction). That is, the performance gap exceeds the accuracy threshold δ between the actual and ideal global model. In this paper, a designed scheme Π should meet the following security goals:

- *Gradients confidentiality:* Without authorization, no client or server can obtain the gradient g_i^k of a specific client c_i $(i \in [1, m])$ during the k-th epoch, even if they collaborate in the collusion attack.
- *Identifying malicious clients:* Malicious clients can be identified without the trusted participants. The server does not collect a small amount of public data from each client for training purposes to verify their gradients.
- *Correctness of the global model:* After identifying malicious clients, FL system should mitigate the impact of incorrect gradients on the global model, avoiding to degrade the performance of global model obviously.

V. Approach Design

We propose an approach called SMTFL, as shown in Fig. 2. To ensure the confidentiality of gradients,We group clients together, using their gradients and gradient shares to conceal each other's gradients. This method allows clients' gradients to protect those of others, mitigating collusion attacks among clients. To identify malicious clients, we assess the impact of the aggregated gradients of a group on the global model, thereby providing a unified evaluation of client behavior. A client that frequently degrades the global model's performance will be classified as malicious. To ensure the model correctness, each client's gradient is encrypted and uploaded to the storage server, where access to and decryption of the gradients require consensus from all clients by providing their respective secret shares. FUL is employed to invalidate the poisoned gradients from the global model, restoring its performance. In this paper, the main notations are summarized in Table II.

A. Gradient Confidentiality Protection

In FL scenarios with untrusted clients and server, where the server is unnecessary to collect a small amount of public data for each client, we aim to reduce the impact of clients' collusion on the global model and identify malicious clients. To achieve this, we group all clients into groups of size $group_n = 3$. Based on the idea of edge computing, these gradients from 3 clients are

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THE MAIN NOTATIONS IN THIS PAPER

Notations	Meanings
$c_i,\ i \in \{1,\ldots,m\}$	The i-th client, and there are m clients in the FL system
s_a	The aggregation server
d_{c_i}	The training data of the c_i
$group_n$	The number of clients in a group.
g_i	The gradient of the c_i.
$\{g_A^1, g_A^2\}$	The gradient shares for the c_A.
$g_{A,B,C}$	The group gradient of the $\{c_A, c_B, c_C\}$.
ε	The perturbation negotiated by the client and the server.
$Model_G$	The global model.
τ	The predefined threshold for the allowable performance change of $Model_G$.
eva_i	The evaluation score for the c_i.
eva_i^k	The evaluation score for the c_i in the k-th epoch.
$thre_{eva}$	The evaluation threshold for determining malicious clients.
$\{P_k, S_k\}$	The public and secret keys.
t	The threshold for Threshold Encryption.
S_i^j	The secret share that client c_i sends client c_j.
d_g	The encrypted data.

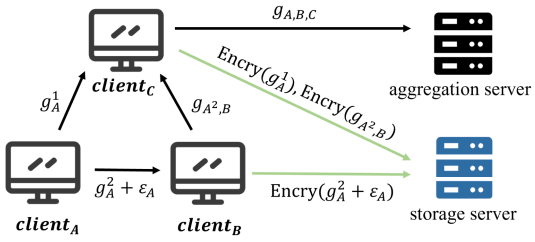


Fig. 3. The illustration of gradient aggregation in one group.

locally aggregated into a group gradient, which is then uploaded to the server by one client. However, untrusted clients and server may leverage gradients to expose the training data of other clients. Therefore, the gradient confidentiality protection must be considered during the group aggregation, as illustrated in Fig. 3.

In protecting gradients, we do not make each client randomly generate noise and add to their local gradient updates (i.e., differential privacy), as this could negatively impact the global model's performance. Nor do we apply homomorphic encryption to the gradients, thereby avoiding complex cryptographic operations. Within a group of clients $\{c_A, c_B, c_C\}$, we randomly split one client's (e.g., c_A) gradient:

$$g_A = g_A^1 + g_A^2 \tag{1}$$

where g_A^1 is randomly generated, and $\{g_A^1, g_A^2\}$ are distinct from g_A. Each client negotiates a dynamic vector ε with the server upon joining the system (e.g., c_A has the ε_A), which is used to obfuscate their gradients.

Specifically, c_A sends its gradient shares $g_A^2 + \varepsilon_A$ and g_A^1 to c_B and c_C, respectively. c_B obfuscates its gradient g_B by $g_A^2 + \varepsilon_A$ and ε_B, and it gets and sends

$$g_{A^2,B} = g_A^2 + \varepsilon_A + g_B + \varepsilon_B \tag{2}$$

to c_C. The group gradient is aggregated by the c_C as

$$g_{A,B,C} = g_A^1 + g_{A^2,B} + g_C + \varepsilon_C \tag{3}$$

Then, $g_{A,B,C}$ is uploaded to s_a. Since s_a knows $\{\varepsilon_A, \varepsilon_B, \varepsilon_C\}$, it can recover the aggregated precise gradient of this group.

After deploying the group aggregation rules, neither any client nor the server can obtain the precise gradients of other clients under normal circumstances: However, certain clients may collude with the server to extract the precise gradients of other clients. For instance, in $\{c_A, c_B, c_C, s_a\}$, exposing the g_A requires the collusion among $\{c_B, c_C, s_a\}$. We observe a fact: during a gradient disclosure operation performed by one party, it may share false gradients to deceive other conspirators. Consequently, three colluding parties will independently disclose g_A. In this collusion:

- c_C obtains $g_A^2 + \varepsilon_A$ and ε_A from c_B and s_a, respectively.
- c_B obtains g_A^1 and ε_A from c_C and s_a.
- s_a obtains g_A^1 and $g_A^2 + \varepsilon_A$ from c_C and c_B, respectively.

All colluding parties independently discloses g_A. However, in this process, c_C also possesses

$$\{g_A^2 + \varepsilon_A + g_B + \varepsilon_B,\ g_A^2,\ \varepsilon_A\}$$

c_C could potentially collude again with s_a to obtain ε_B, thereby acquiring g_B. Thus, c_B is unwilling to collude with c_C and s_a. The collusion revealing g_B and g_C follows the similar pattern, we will not repeat the description.

In other words, *three clients impose constraints on one another in one group:* Honest clients will refrain from colluding to reveal other clients' gradients, avoiding their own gradients be exposed as well. It is noteworthy that malicious clients may willingly participate in collusion to reveal the gradients of other clients. Once a client engages in collusion, it indicates that this client has executed a poisoning attack, and its gradients are erroneous or fabricated. *We find that the semi-honest server is a necessary participant in collusion*, it can penalize collusion clients to mitigate the effects of poisoning attacks from malicious clients.

Above, we achieve gradient aggregation and privacy protection among a group of clients locally, alleviating the computational burden on the server. We consider that a group size $group_n = 3$ is a reasonable value. First, when the $group_n = 2$, a client can easily collude with the server to expose another client's gradient. Then, we want to locate malicious clients in a smaller $group_n$ through monitoring the behaviors of each client. All clients are grouped dynamically in each epoch. If the number of clients falls below 3in a group, this group will be temporarily excluded from updates to the global model.

B. Malicious Clients Identification

In assessing the correctness of gradients, we do not require the server s_a to collect a small amount of public data for each client for training, nor do we rely on the assumption that the gradient direction based on the majority of clients is correct, as this assumption carries the risk of collusion attacks. Instead, for a group of clients $\{c_A, c_B, c_C\}$, we evaluate the impact of their aggregated group gradient $g_{A,B,C}$ on the global model's performance, then *provide a unified evaluation for their behaviors:* Based on the group gradient, the evaluation method realizes that there is more training data to compute the local gradient updates

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.

and to affect the global model more effectively, as shown in the relevant part of Fig. 2.

We describe the unified evaluation rule for a group of clients. Specifically, for the $\{c_A, c_B, c_C\}$, each client is assigned an evaluation score $\{eva_A, eva_B, eva_C\}$. Their group gradient $g_{A,B,C}$ is aggregated to the global model $Model_G$ in the k-th epoch. Compared to the $(k-1)$-th epoch, there are three possible scenarios:

- If the performance degradation of $Model_G$ exceeds a predefined threshold τ, this is,
$$Per_{k-1} - Per_k > \tau \tag{4}$$
a poisoning attack event is suspected, where Per_k denotes the performance of the $Model_G$ at the k-th epoch. Each client is assigned a score $eva_i^k = +1$ in the k-th epoch, where $i \in \{A, B, C\}$.
- If the performance change of $Model_G$ falls within an acceptable range $[-\tau, \tau]$, that is,
$$|Per_k - Per_{k-1}| < \tau \tag{5}$$
each client is assigned a score of $eva_i^k = 0$ in the k-th epoch, where $i \in \{A, B, C\}$. This setting allows for a tolerance of non-malicious, occasional errors, such as low data quality or natural training fluctuations.
- If the performance improvement of $Model_G$ exceeds the predefined threshold τ, that is,
$$Per_k - Per_{k-1} > \tau \tag{6}$$
we consider that no poisoning event is detected, and each client is assigned a score of $eva_i^k = -1$ in the k-th epoch, where $i \in \{A, B, C\}$.

The cumulative evaluation for each client is the sum of the scores assigned over multiple epochs:

$$eva_i = \sum eva_i^k, \quad i \in \{A, B, C\} \tag{7}$$

The client is malicious if satisfying $eva_i < thre_{eva}$. In other words, *a client c_i is considered as malicious if its eva_i falls below a preset threshold $thre_{eva}$, and it will be removed from the system:* Its historical gradients are invalidated from $Model_G$ to ensure the model's performance (see Section V-C).

It is noted that while these clients are uniformly scored in the same group, not all clients may be malicious, which could lead to false positives. Assigning a score of $eva_i^k = 1$ helps mitigate the impact on mistakenly targeted clients. Dynamic client grouping can reduce the effect on such clients. To achieve more precise identification of malicious clients, 3 clients from the same group are intentionally distributed into different groups in subsequent epochs.

C. Model Correctness Improvement

Some poisoning gradients from malicious clients may be aggregated into the global model for several reasons:

- A group consists of both honest and malicious clients. Due to the influence of the honest clients' gradients, their group gradient may not lead to a significant drop in the performance of the global model.
- We evaluate whether a client is malicious based on its cumulative evaluation score.

Therefore, if the model performance oscillates within the range $[-\tau, \tau]$ or significantly deteriorates, s_a will notify the clients in this group to encrypt the received gradients or gradient shares (collectively referred to as gradients) and upload them to a storage server. Once a client is identified as malicious (see Section V-B), its aggregated historical gradients are decrypted. Then, we employ the FUL method invalidate these gradients in the global model. The role of the storage server is necessary:

- If each client stores its own gradients, a malicious client might refuse to reveal its poisoning gradients.
- If gradients are stored by clients performing the aggregation, their limited resources may not support storing a large number of historical gradients.
- Clients dynamically join or leave the FL system, potentially preventing the retrieval of poisoning gradients.

Although gradients are encrypted and stored on the storage server, all participants are untrusted in the system, as they all want to access other clients' training data. A critical issue is that the semi-honest server can arbitrarily decrypt and access any client's precise gradient if it independently holds the decryption key for the gradients. This issue seriously compromises the confidentiality of gradients. Therefore, *we adhere to the idea of multi-party governance, where the decryption key is not held by a single party:* Based on the idea of threshold encryption, s_a initiates a decryption request, all clients participate in consensus-based decryption. Specifically, it can be divided into three parts: key generation, encryption and decryption of gradients, and gradient forgetting.

In key generation, although each client's gradients are sent to the storage server, they only need to be confidential from other clients and the storage server. The client will not collude with others to decrypt others' gradients. As shown in Fig. 4, we design that each client c_i $(i \in \{1, \ldots, m\})$ generates its public-private key pair $\{P_i, S_i\}$. The public key P_i is used to encrypt gradients transmitted to the storage server. The private key S_i forms a threshold encryption scheme $\binom{m}{t}$, and it is divided into $(m-1)$ key shares based on threshold encryption:

$$\{S_i^1, \ldots, S_i^{m-1}\}$$

at least t shares are required to recover S_i. We generate each share S_i^j based on the Shamir scheme, which generates a random polynomial:

$$f(x) = S + a_1 x + a_2 x^2 + \cdots + a_{t-1} x^{t-1} \pmod p \tag{8}$$

For $\forall$ client c_j $(j \in \{1, \ldots, i-1, i+1, \ldots, m\})$, it sends a random number r_j to the client c_i, and c_i inputs the r_j into the $f(x)$ to obtain the c_j's key share S_i^j:

$$S_i^j = (r_j, f(r_j)) \tag{9}$$

Thus, when s_a requests to decrypt the detected poisoning gradient g_m, at least t clients join in the decryption, *even if a few clients dynamically join or exit the system without providing their shares:*

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.

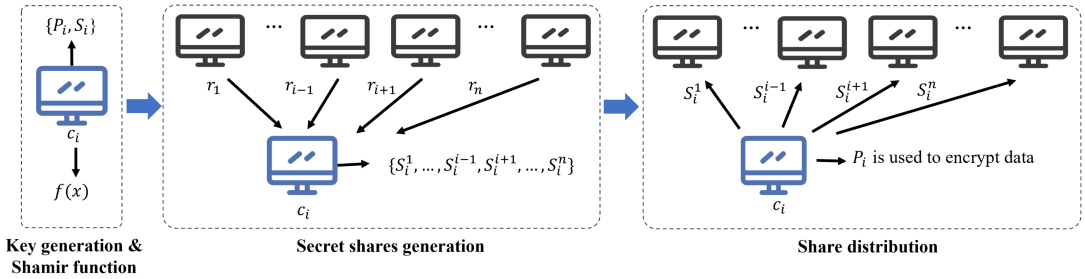


Fig. 4. The generation and distribution of secret shares.

During the process of encrypting gradients and sending them to the storage server, $\forall\ c_i$ uses its public key P_i to encrypt the received gradient d_g (e.g., g_A^1, $g_{A^2,B}$ or $g_A^2 + \varepsilon_A$):

$$Data_{decryed}^{c_i} = Encry_{P_i}(d_g) \tag{10}$$

Then, c_i sends $Data_{decryed}^{c_i}$ to the storage server.

After s_a has identified the malicious c_m (see Section V-B), it requests the storage server for the poisoning g_m of c_m.

Storage server requests the key shares from all online clients, and at least t clients send some clients' key shares, where these clients has cooperating with c_m. For instance, the c_i has cooperating with c_m, its shares $\{S_i^1, \ldots, S_i^t\}$ is received. The key S_i is recovered using Lagrange interpolation function:

$$S_i = \sum_{j=1}^{t} f(r_j) \prod_{1 \le l \le t, l \ne j} \frac{r_l}{r_l - r_j} \pmod p \tag{11}$$

s_a uses the S_i to decrypt the d_g, that is related g_m, from other clients. Meanwhile, s_a possesses ε_{c_m} and obtains the g_m.

After obtaining the poisoning gradients $\{g_m^1, \ldots, g_m^k\}$ from malicious clients $\{c_m^1, \ldots, c_m^k\}$, the server applies FUL to invalidate their impact on the global model. Since FUL itself is not an innovation of this work, we utilize it to mitigate the influence of poisoning gradients and restore the accuracy of the global model. To achieve this, we implement FUL by directly removing the already aggregated gradients:

$$g_{global} = \frac{\sum g_{group} - \sum_{j=1}^{k} g_m^j * \omega_{c_m^j}}{m - k} \tag{12}$$

Where g_{group} denotes the sum of gradients from all participating clients, g_m^j represents the poisoning gradient from the malicious client c_m^j, and $\omega_{c_m^j}$ is the weight assigned to the malicious client's gradient, typically adjusted based on the severity of the attack or other evaluation metrics. m denotes the total number of clients in the federation, while k represents the number of identified malicious clients. At this point, *we mitigate the impact of the aggregated poisoning gradients on the global model:*

VI. Security Analysis

Theorem 1: Let a group contain 3 clients $\{c_A, c_B, c_C\}$. Each client c_i generates secret shares. No client or server can individually obtain the precise gradients of other clients. Meanwhile, any two semi-honest clients will not collude with the aggregation server s_a to obtain the precise gradient of another client. The group's clients will not collude to launch a poisoning attack.

Proof: The client c_A transmits the masked shares g_A^1 and $g_A^2 + \varepsilon_A$. The client c_B transmits the masked share

$$g_{A^2,B} = g_A^2 + \varepsilon_A + g_B + \varepsilon_B \tag{13}$$

The client c_C transmits the masked share

$$g_{A,B,C} = g_A^1 + g_{A^2,B} + g_C + \varepsilon_C \tag{14}$$

Consider the adversaries c_B or c_C while c_A is honest. They observe g_A^1 and $g_A^2 + \varepsilon_A$ and do not know ε_A, which are independently generated and agreed between c_A and s_a. Thus, g_A is confidential. For the c_C, it is unable to infer g_B from $g_{A^2,B}$ because g_A^2, ε_A, and ε_B remain unknown. For the aggregation server s_a, it only obtains the group-level gradient $g_{A,B,C}$ under normal operation.

The storage server keeps the encrypted historical gradients, whose decryption requires consensus from a majority of clients. Specifically, when the number of clients agreeing to decrypt does not meet the predefined threshold $\binom{m}{t}$, the decryption process cannot be performed. This establishes a security boundary: as long as the number of malicious clients does not exceed t, the storage server is unable to obtain any client's gradient.

Therefore, no client or server can individually obtain the precise gradients of other clients.

Collusion with the semi-honest server s_a does not leak additional information, since s_a only receives the same pair of masked shares. In this collusion among c_B, c_C, and s_a, they exchange the g_A^1, $g_A^2 + \varepsilon_A$, and ε_A, respectively. As a result, c_B, c_C, or s_a can induce g_A:

$$g_A = g_A^1 + g_A^2 + \varepsilon_A - \varepsilon_A \tag{15}$$

However, in this scenario, c_B's gradient g_B will also be revealed by the collusion between c_C and s_a. specifically, c_C gets the ε_B from s_a and induce g_B:

$$g_B = g_{A^2,B} - g_A^2 + \varepsilon_A - \varepsilon_B \tag{16}$$

That is, g_B is revealed. Thus, semi-honest clients will not collude, as their gradients would also be exposed.

Regarding poisoning attacks, SMTFL determines whether to aggregate a group gradient based on its impact on the global model. In a group of clients, if two or more clients launch poisoning attacks, the incorrect gradients will dominate the

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.

direction of the group's gradient, thereby degrading the global model's performance. According to the rule in Section V-B, such a group's gradient will not be integrated into the global model. Therefore, no rational client will collude. □

Privacy Leakage Upper Bound: Since SMTFL employs gradient splitting and intra-group obfuscation, each client can observe only a single masked gradient share. Let the original gradient be g, and let the observed masked share be denoted as $s_{mask} = g' + \varepsilon$, where g' is a gradient share (e.g., g_A^2) and ε represents the random perturbation vector. To strictly quantify the leakage, we introduce the following assumption regarding the noise distribution:

Distribution assumption (High-Entropy Obfuscation): We assume that the perturbation ε is drawn from a distribution with sufficiently high entropy. Specifically, ε follows either a uniform distribution over a finite field $\mathbb{F}p$ or a high-variance Gaussian distribution $\mathcal{N}(0, \sigma^2 I)$. Under this assumption, the probability distribution of s_{mask} is dominated by ε.

Under this assumption, the masked share is statistically indistinguishable from random noise for any adversary who does not possess ε. Formally, the conditional entropy of the observed share given the gradient depends only on the noise:

$$H(s_i|g) = H(g + \varepsilon|g) = H(\varepsilon) \tag{17}$$

Furthermore, due to the high entropy of ε, the marginal entropy of the share approximates that of the noise: $H(s_{mask}) \approx H(\varepsilon)$.

Consequently, the mutual information satisfies:

$$I(g'; s_{mask}) = H(s_{mask}) - H(s_{mask}|g') \approx H(\varepsilon) - H(\varepsilon) = 0 \tag{18}$$

This derivation confirms that, information-theoretically, the observation of a single masked share s_i reveals negligible information about the specific component of the true gradient g, ensuring privacy against the defined semi-honest adversary.

Theorem 2: Consider each client c_i maintaining a cumulative evaluation score

$$eva_i = \sum_{k=1}^{K} s_i^k \tag{19}$$

where $s_i^k \in \{-1, 0, +1\}$ indicates whether the unified evaluation of the aggregated group gradient at epoch k improves (-1), change at range $[-\tau, \tau]$ (0) or degrades $(+1)$ the performance of $Model_G$.

Assume that honest clients contribute positively in most epochs, while malicious clients intentionally degrade performance. Let $thre_{eva}$ be a threshold chosen such that it lies between the cumulative scores of honest and malicious clients over K epochs. Then after K epochs

$$eva_H < thre_{eva} < eva_M \tag{20}$$

for all malicious clients c_M and honest clients c_H, assuming that honest clients' aggregated contributions dominate the malicious ones. Consequently, malicious clients can be reliably separated from honest clients based on their cumulative evaluation scores.

Proof: Each client's cumulative evaluation score eva_i is an integer sum of $+1$, 0 and -1 over K epochs.

- For honest clients, eva_H tends to decrease, because in most epochs their inclusion in the group leads to a performance improvement of $Model_G$, i.e., $s_H^k = -1$.
- For malicious clients, eva_M tends to increase, because they inject poisoning gradients that degrade the global model, i.e., $s_M^k = +1$.

By summing over K epochs, the cumulative scores of honest clients become larger than those of malicious clients. Selecting the threshold $thre_{eva}$ between these two ranges ensures

$$eva_H < thre_{eva} < eva_M \tag{21}$$

Thus, malicious clients are identifiable and removed from the system. The separation becomes clearer as K increases. □

Adaptive Attack Analysis: Adaptive attackers may occasionally behave as honest clients to evade detection, thereby prolonging the time required for the system to identify them. However, such behavior reduces the attacker's per-epoch effectiveness in degrading the global model.

Let Δ_{cum} denote the cumulative performance degradation caused by a malicious client before detection. We define γ as the minimum net performance decrease caused by the attacker in any epoch, accounting for the mitigation effects of honest clients within the dynamically formed group. That is,

$$\gamma = \min_k \left(\Delta_k^{\text{attacker}}\right) \tag{22}$$

Since the detection removes a client once its cumulative evaluation score eva_i falls below the threshold $thre_{eva}$, a conservative upper bound on the damage is given by:

$$\Delta_{\text{cum}} \leq \tau \cdot \left\lceil \frac{thre_{eva}}{\gamma} \right\rceil \tag{23}$$

where τ is the predefined maximum per-epoch performance change considered in the unified evaluation.

Even under adaptive attacks, this bound shows that the total damage a malicious client can inflict is limited. The dynamic grouping of clients and the unified cumulative scoring mechanism ensure that attackers cannot indefinitely evade detection: while occasionally pretending to be honest delays the identification, it simultaneously reduces the attacker's impact in each epoch. Consequently, SMTFL effectively constrains the cumulative influence of adaptive attackers on the global model while still reliably identifying malicious clients.

Theorem 3: Threshold encryption secures the gradients storage from all clients and servers, and it allows for the effective retrieval of poisoning gradients, even when some clients dynamically join or leave the system.

Proof: In a system with m clients, each client possesses their own public and private keys. For instance, client c_i uses its public key P_i to encrypt the received gradient data $Encry_{P_i}(g)$. Its private key S_i is split into $m-1$ shares S_i^j ($j \in \{1, \ldots, i-1, i+1, \ldots, m\}$) through threshold encryption mechanisms and sent to the other $m-1$ clients. c_i holds the encrypted data, and no other client or servers can possess the complete S_i, achieving secure storage of gradients. During the decryption of $Encry_{P_i}(g)$, SMTFL does not require c_i to perform decryption operations. The s_a sends requests for the key shares of S_i to all online clients. As long as at least t clients response, the s_a

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.

can recover S_i and decrypt the poisoning gradients of malicious clients, even if c_i or a few other clients are offline. □

Theorem 4: Let the system store encrypted gradients g_i using a $\binom{m}{t}$ threshold scheme with public and secret keys $\{P_k, S_k\}$. Assume an adversary controls fewer than t clients, so it cannot decrypt g_i or modify historical information. Suppose FUL identifies a set of poisoned historical gradients $\mathcal{P}$ and removes them from reconstruction of $Model_G$. Let the reconstructed clean update be $G_{\text{clean}} = \sum_{c_i \notin \mathcal{P}} g_i$. Then the recovered model after K epochs satisfy $\|Model_G^K - Model_G^\star\| \leq O(\frac{|\mathcal{P}|}{K})$, where $Model_G^\star$ is the stationary point obtained under clean training.

Proof: Because the adversary controls fewer than t clients, it cannot obtain the necessary shares to decrypt g_i. Thus, the system can reconstruct all historical clean gradients after removing the identified poisoned set $\mathcal{P}$. Let the deviation introduced by poisoned gradients be $\Delta_{\mathcal{P}} = \sum_{c_i \in \mathcal{P}} g_i$.

Before rollback, the model follows the perturbed path:

$$Model_G^{k+1} = Model_G^k - \eta(G_{\text{clean}} + \Delta_{\mathcal{P}}) \tag{24}$$

After rollback, the model follows:

$$Model_G^{k+1} = Model_G^k - \eta G_{\text{clean}}. \tag{25}$$

Because gradient descent converges at $O(\frac{1}{K})$, and the poisoning perturbation magnitude averages out as

$$\frac{\|\Delta_{\mathcal{P}}\|}{K} = O\left(\frac{|\mathcal{P}|}{K}\right) \tag{26}$$

$$\|Model_G^K - Model_G^\star\| \leq O\left(\frac{|\mathcal{P}|}{K}\right) \tag{27}$$

Hence, SMTFL guarantees stable recovery of $Model_G$ even under multi-epoch adaptive poisoning. □

VII. Experiment Evaluation

A. Experimental Setting

Environment: We implement the experiments on a Linux server equipped with 1 NVIDIA RTX A6000 GPU. The experiments utilize the PyTorch framework, with CUDA version 11.4, Torch version 1.10.1+cu111, and the Python 3.6.13.

Datasets and partitioning: We evaluate SMTFL in image classification and object detection tasks using five datasets: MNIST [31], Fashion-MNIST (FMNIST) [32], Extended MNIST (EMNIST) [33], CIFAR-10 [34], and Street View House Number (SVHN) [35]. Datasets are split into training and test sets at an 8:2 ratio, where the training set is used for local client training, and the test set is used to evaluate the global model's accuracy. These datasets are widely used as benchmark datasets in existing work, particularly in scenarios involving poisoning attacks. Following the prior works [17], [36], [37], We simulate the iid rate ($rate_{iid}$) by partitioning each client's dataset into label-specific subsets. For each client, a portion of samples is randomly selected from a target label l at a proportion of $rate_{iid}$, while the remaining samples are randomly selected from other labels at a proportion of $(1 - rate_{iid})$.

Global models: To show SMTFL's generalizability, we evaluate it by different combinations of datasets and models. The evaluation is conducted on MNIST, EMNIST, and FMNIST using a Convolutional Neural Network (CNN) and on CIFAR-10 and SVHN using a Residual Neural Network (ResNet-18).

Baseline attacks: We evaluate SMTFL against both poisoning attacks and gradient inversion attacks. For the poisoning attacks, we consider three common scenarios:

- *Label Flipping Attack [38]:* Untrusted participants arbitrarily modify the labels of training data based on the gradients uploaded by the clients.
- *Random Update Attack [24]:* Malicious clients arbitrarily alter the uploaded gradients, leading to poor aggregation results for the global model and causing it to deviate from normal performance.
- *Projected Gradient Descent Attack (PGD) [39]:* Malicious clients utilize gradient information to generate adversarial samples that are visually similar to normal samples. However, these adversarial samples cause global model to make incorrect predictions.

For the gradient inversion attacks, we consider to implement the image reconstruction attacks [40], where malicious participants attempt to reconstruct the training data based on the obtained gradients.

Baseline defenses: To demonstrate SMTFL's advantage, we compare it with existing FL aggregation methods, including *Fedavg* [41], *Krum* [23], *Cos Defense* [42], and median/mean-based approaches such as *Median*, *Clipping Median* and *Trimmed Mean* [24]. Additionally, we compare SMTFL with the recently proposed *DDFed* [18], which addresses the same issues as our work, namely protecting training data privacy and defending against poisoning attacks.

Parameter settings: Unless otherwise specified, the default number of clients is set to 150, with the percentage of malicious clients ($Prop_{MC}$) set to 25% . The data distribution is controlled by $rate_{iid} = 0.5$. We explore the SMTFL's effectiveness against the baseline attacks under different proportions of malicious clients and data distributions. Meanwhile, we compare it with baseline defenses to demonstrate its advantages. Table III shows the default parameters settings, which will be employed unless otherwise mentioned.

Evaluated parameters, including:

- *Defense against gradient poisoning attacks:* The effectiveness of the defense is evaluated based on the performance changes of the global model. This evaluation includes the model's performance before and after poisoning attacks, as well as the performance changes after deploying SMTFL. The model performance is measured in terms of accuracy.
- *Defense against gradient inversion attacks:* We compare the gradients and the reconstructed visual data before and after deploying SMTFL.
- *Influence of $rate_{iid}$:* The effectiveness of SMTFL in defending against poisoning attacks is evaluated under different $rate_{iid}$ settings.
- *Detection of malicious clients:* We focus on identifying and removing malicious clients. Specifically, we evaluate SMTFL's ability to detect malicious clients and the number of epochs required for detection. The detection accuracy

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.

TABLE III
PARAMETERS SETTINGS

Dataset	num_{class}	Model	num_{client}	$rate_{iid}$	$thre_{eva}$	$Epoch_{Local}$	$Epoch_{Global}$	Batchsize	$Prop_{MC}$
MNIST	10	CNN	150	0.5	6	20	30	100	25%
FMNIST	10	CNN	150	0.5	6	20	30	100	25%
CIFAR-10	10	ResNet-18	150	0.5	6	50	30	100	25%
EMNIST	47	CNN	150	0.5	6	20	30	100	25%
SVHN	10	ResNet-18	150	0.5	6	20	30	100	25%

(acc_{loc}) is defined as the proportion of identified malicious clients (num^{md}_{client}) to the total number of malicious clients ($Prop_{MC} \times num_{client}$).

- *False positive rate ($rate_{False}$) for malicious clients:* When evaluating a group of clients uniformly, an honest client may be mistakenly classified as malicious. $rate_{False}$ is defined as the proportion of honest clients misclassified as malicious (num^{kd}_{client}) relative to the total number of honest clients $(1 - Prop_{MC}) \times num_{client}$.
- *Encryption overhead:* We evaluate the time taken to generate shares in threshold encryption, as well as the time needed for gradient encryption and decryption.
- *Storage overhead:* Since the storage server stores historical gradients from clients, we evaluate the storage space it needs to provide for each client.

B. Experimental Results

We perform a comprehensive evaluation of SMTFL based on the evaluated parameters outlined above.

Effectiveness of SMTFL: We conduct a thorough evaluation of SMTFL on datasets using the two types of baseline attacks described above.

- *Defense against poisoning attacks:* Under a fixed setting of 150 clients, we consider scenarios where the number of malicious clients does not exceed half of the total population (i.e., $\frac{m}{2}$). SMTFL is tested under varying proportions of malicious clients ($Prop_{MC}$) to assess its robustness. As shown in Table IV, we report the changes in global model accuracy before and after poisoning attacks across different $Prop_{MC}$ values (5% , 15% , 25% , 35% , 45%), along with the corresponding defense effectiveness achieved by SMTFL. Notably, we observe a slight increase in model accuracy when $Prop_{MC}$ is relatively small (e.g., around 5%). We attribute this to a potential improvement in model generalization under mild perturbations, which may, to some extent, enhance performance on downstream tasks.
- *Defense against gradient inversion attacks:* Semi-honest servers and clients can initiate gradient inversion attacks. Since SMTFL groups clients together, there are four stages in the gradient transmission process when clients upload gradient data to the server (e.g., g^1_A, $g^2_A + \varepsilon_A$, $g_{A^2,B}$, and $g_{A,B,C}$). Semi-honest servers and clients can attempt to access client's gradient data at each of these stages to perform gradient inversion attacks, aiming to reconstruct training data and compromise client's privacy. To assess the effectiveness of SMTFL against such attacks, we conducted experiments at each transmission stage. The results, shown in Fig. 5, demonstrate that servers and clients are unable to reconstruct client's training data from gradients acquired at any stage, thus protecting client's privacy.

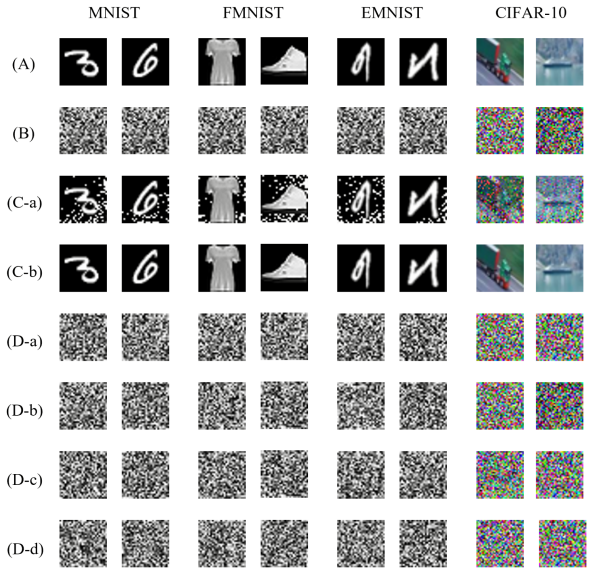


Fig. 5. Effectiveness of SMTFL in defense against gradient inversion attacks. (A) the original images; (B) the attacked images; (C) the reconstructed images are obtained through gradient inversion attacks during the 5-th and 10-th epochs, which means that the image can be reconstructed in a few epochs; (D) the reconstructed images obtained through gradient inversion attacks when SMTFL is deployed, where images are reconstructed through $\{g^1_A, g^2_A + \varepsilon_A, g_{A^2,B}, g_{A,B,C}\}$.

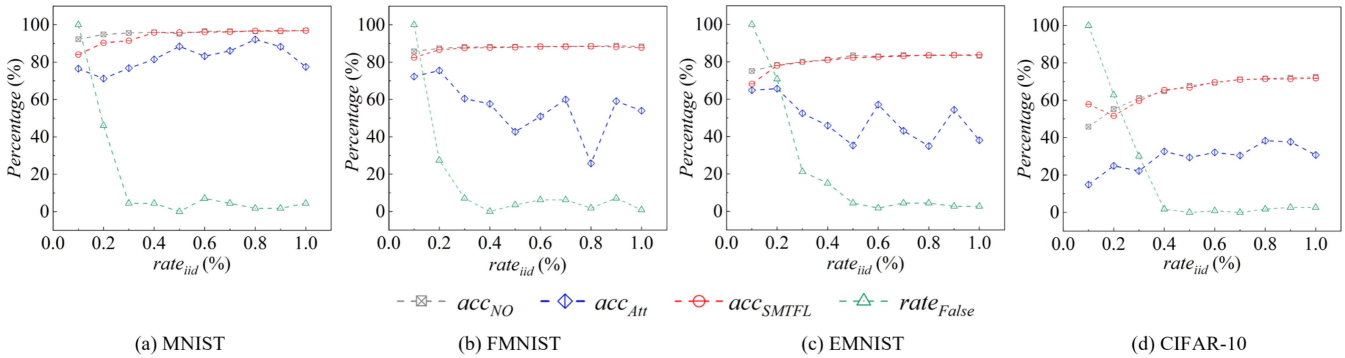
Parameter sensitivity analysis: To assess SMTFL's robustness and generalization ability under different parameter settings, we evaluate the performance of SMTFL, including the model accuracy, the proportion of correctly identified malicious clients, and the false positive rate for honest clients.

- *Impact of $rate_{iid}$:* We evaluate the performance of SMTFL on four datasets (i.e., MNIST, FMNIST, EMNIST, and CIFAR-10) with different $rate_{iid}$, as shown in Fig. 6. The results demonstrate that SMTFL accurately identifies malicious clients across different data distributions, achieving a proportion of correctly identified malicious clients (the ratio of detected to actual malicious clients) over 97.3% . With deploying SMTFL, the model's accuracy remains largely on par with or slightly lower than that of the model without any attacks. When the $rate_{iid}$ is low (e.g., $rate_{iid} \leq 0.2$), the highly imbalanced data distribution leads to a higher number of honest clients being

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
THE CHANGE IN MODEL ACCURACY BEFORE AND AFTER DEPLOYING SMTFL UNDER DIFFERENT ATTACKS AND $Prop_{MC}$

Dataset	Model	No attack	num_{client}	Attack method	Stage	Malicious clients proportions & Model accuracy				
						5%	15%	25%	35%	45%
MNIST	CNN	95.23%	150	PGD	Attacked (%)	94.87	13.96	9.80	9.83	9.77
					Defended (%)	95.64	96.28	96.65	96.22	95.56
				Label Flip	Attacked (%)	96.52	95.77	89.07	88.34	86.81
					Defended (%)	96.53	96.92	97.09	96.52	96.60
				Random Update	Attacked (%)	95.84	95.29	88.42	59.71	25.60
					Defended (%)	95.85	96.11	95.92	95.93	95.59
CIFAR-10	Resnet-18	67.78%	150	PGD	Attacked (%)	77.59	73.68	71.17	57.30	53.34
					Defended (%)	78.13	76.78	76.95	64.06	61.90
				Label Flip	Attacked (%)	68.35	73.75	55.30	46.45	37.88
					Defended (%)	69.44	78.01	67.71	76.58	65.34
				Random Update	Attacked (%)	73.26	49.81	29.38	20.60	16.30
					Defended (%)	73.31	73.11	73.85	66.57	66.80
FMNIST	CNN	88.19%	150	PGD	Attacked (%)	83.85	86.33	86.59	80.60	76.62
					Defended (%)	88.04	86.36	96.66	87.09	86.71
				Label Flip	Attacked (%)	86.65	81.23	46.51	28.44	11.15
					Defended (%)	87.77	88.04	87.49	87.51	87.54
				Random Update	Attacked (%)	87.52	76.53	62.68	35.36	10.00
					Defended (%)	88.00	88.27	88.25	87.50	87.47
EMNIST	CNN	83.43%	150	PGD	Attacked (%)	75.78	64.27	57.20	9.97	2.14
					Defended (%)	80.75	82.70	81.90	81.58	81.18
				Label Flip	Attacked (%)	82.38	81.51	80.76	79.44	74.23
					Defended (%)	82.77	82.11	82.88	82.45	81.16
				Random Update	Attacked (%)	82.82	80.12	35.22	27.97	18.36
					Defended (%)	83.03	82.11	82.22	81.91	80.92
SVHN	Resnet-18	91.27%	150	PGD	Attacked (%)	90.85	63.89	36.52	19.57	15.91
					Defended (%)	91.32	90.26	90.82	89.27	89.67
				Label Flip	Attacked (%)	88.65	61.73	37.45	15.92	9.68
					Defended (%)	90.92	89.04	88.49	88.76	88.55
				Random Update	Attacked (%)	89.67	68.01	49.38	45.39	39.67
					Defended (%)	91.85	89.28	88.96	88.03	90.25



(a) MNIST (b) FMNIST (c) EMNIST (d) CIFAR-10

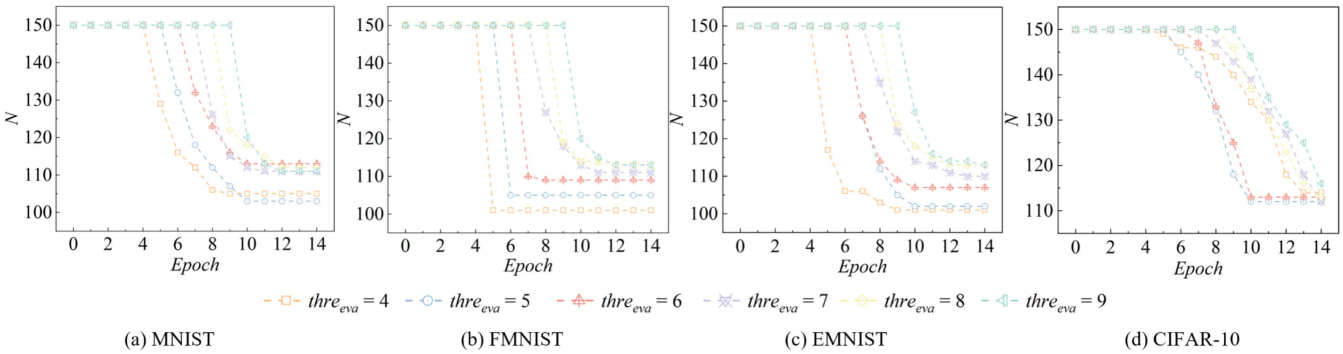
Fig. 6. Model accuracy after deploying SMTFL (acc_{SMTFL}), model accuracy without attack (acc_{No}), model accuracy after being attack (acc_{Att}), and the $rate_{False}$ across four datasets with different $rate_{iid}$.

misclassified as malicious. This occurs because, in extreme Non-IID scenarios (e.g., low $rate_{iid}$), the local gradient updates of honest clients may differ significantly from the global gradient direction. Since SMTFL utilizes a unified evaluation mechanism that assesses the impact of aggregated gradients on the global model's performance, these updates are identified as causing performance degradation and are thus penalized. While techniques for personalized FL could mitigate such effects caused by data heterogeneity, they remain outside the scope of this paper. Despite the higher $rate_{False}$, Fig. 6 demonstrates that SMTFL effectively restores model accuracy to levels close to the

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.

TABLE V
SMTFL'S PERFORMANCE UNDER DIFFERENT THRESHOLD SETTINGS, INCLUDING MODEL ACCURACY (acc_{SMTFL}), MALICIOUS CLIENT LOCALIZATION ACCURACY (acc_{loc}), AND FALSE POSITIVE RATE ($rate_{False}$)

$thre_{eva}$	MNIST			FMNIST			EMNIST			CIFAR-10		
	acc_{SMTFL}	acc_{loc}	$rate_{False}$	acc_{SMTFL}	acc_{loc}	$rate_{False}$	acc_{SMTFL}	acc_{loc}	$rate_{False}$	acc_{SMTFL}	acc_{loc}	$rate_{False}$
4	96.16%	100.00%	7.08%	86.82%	100.00%	10.62%	82.12%	100.00%	10.62%	65.46%	95.00%	0.88%
5	96.35%	100.00%	8.85%	87.06%	100.00%	7.08%	81.94%	100.00%	9.73%	66.93%	100.00%	0.88%
6	**95.92%**	**100.00%**	**0.00%**	**87.95%**	**100.00%**	**3.54%**	**83.43%**	**100.00%**	**4.42%**	**67.78%**	**100.00%**	**0.00%**
7	96.00%	100.00%	1.77%	87.90%	100.00%	1.77%	82.40%	100.00%	2.65%	67.11%	100.00%	0.88%
8	96.03%	100.00%	0.88%	88.20%	100.00%	0.00%	82.71%	100.00%	0.00%	66.97%	100.00%	1.00%
9	96.10%	100.00%	1.77%	88.16%	100.00%	0.00%	81.91%	100.00%	0.88%	66.71%	97.30%	1.77%



(a) MNIST (b) FMNIST (c) EMNIST (d) CIFAR-10

Fig. 7. The trend of the number of clients participating in the FL system across different $thre_{eva}$ and epoch, on four datasets.

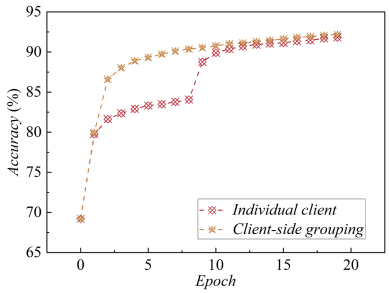


Fig. 8. The trend of model accuracy with respect to the number of rounds, under the methods of individual client gradient aggregation and grouped gradient aggregation.

pre-attack state, confirming that this approach successfully safeguards system robustness against poisoning.

- *Impact of $thre_{eva}$:* To enhance the robustness of FL systems, SMTFL effectively removes malicious clients c_m by setting a threshold $thre_{eva}$. This paper evaluates the model's performance and computational overhead under different threshold settings $thre_{eva}$. The results on model performance are presented in Table V. The findings demonstrate that, regardless of the specific threshold setting, the SMTFL can efficiently identify and remove malicious clients, thereby restoring model accuracy, with the detection accuracy (acc_{loc}) for malicious clients is upper 95% . As the $thre_{eva}$ increases, the $rate_{False}$ for honest clients generally decreases, a trend particularly notable in the FMNIST dataset. For the other three datasets, although some fluctuations in false positives were observed, the overall trend is consistent. However, increasing the threshold is not always the optimal choice, as higher threshold values can prolong the persistence of malicious clients in the system, significantly increasing computational overhead. We further analyze, under different threshold settings across four datasets, the number of clients in the system after continuously detecting and removing malicious clients in each epoch. The results, shown in Fig. 7, indicate that higher threshold values slow down the removal of malicious clients, thereby adding to the system's computational burden. Setting the threshold requires balancing performance and overhead to achieve the optimal trade-off. When $thre_{eva} = 6$, the model achieves the optimal recovery accuracy with minimal adverse effects on honest clients.

Grouping strategy provides extra advantages for model training: SMTFL's grouping strategy not only effectively protects client privacy and prevents gradient leakage but also offers certain advantages for model training. To better demonstrate the benefits of our grouping strategy in improving model performance and convergence speed, we conducted a comparative analysis on the MNIST dataset. Specifically, we compared the trend of model accuracy over training rounds between two approaches: training with individual client gradient uploads to the server and training with grouped client gradients aggregated and uploaded to the server. The results, as shown in Fig. 8, indicate that gradient aggregation from grouped clients leads to superior convergence performance compared to individual client uploads. Furthermore, grouped client aggregation enhances model performance more effectively than individual client aggregation. By training the same model on datasets of varying sizes and comparing their performance improvements, we observed that when a client with a larger dataset participates in the FL system,

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
THE COMPARISON OF COMPUTATIONAL AND COMMUNICATION COMPLEXITY

Method	Computational complexity	Communication complexity	Cryptographic Overhead	Accuracy Impact
FedAvg	$O(d)$	$O(d)$	None	None
HE-based Secure Aggregation	$O(\text{poly}(d,\lambda))$	$O(d \cdot \text{poly}(\lambda))$	Homomorphic encryption operations	None
MPC-based Secure Aggregation	$O(d)$	$O(kd)$	k-party secure protocol	None
DP-based Methods	$O(d)$	$O(d)$	Noise injection	Accuracy degradation
SMTFL (ours)	$\boldsymbol{O(d)}$	$\boldsymbol{O(d)}$	Threshold key sharing	None

TABLE VII
THE STORAGE OVERHEAD OF THE MODEL AND THE ENCRYPTION/DECRYPTION TIME OVERHEAD

Dataset	$Space_{sto}$	m	t	T_{enc} (s)	T_{dec} (s)	T_{gks} (s)
MNIST	0.51 MB	149	90	0.0022	0.0056	0.0044
FMNIST	0.69 MB	149	90	0.0024	0.0057	0.0042
CIFAR-10	256.37 MB	149	90	0.8307	0.8215	0.0042
EMNIST	2.01 MB	149	90	0.0078	0.0084	0.0042

the model's generalization ability is enhanced. Similarly, our grouping strategy, through gradient aggregation, has proven to be an effective method for improving the model's generalization performance. In contrast, clients with smaller datasets may experience gradient updates that deviate from the normal range, potentially affecting the overall model's performance.

C. Complexity Analysis and Storage Overhead

In the SMTFL, we use a threshold encryption to encrypt and store the gradient data generated during the model training process. The number of clients m is 150, and the threshold t is 90. We assess the time overhead associated with this encryption method, which includes the time for encryption, decryption, and generation of key shares, that is, T_{enc}, T_{dec}, and T_{gks}. Concurrently, we evaluate the storage space required by the storage server for each client. The results are detailed in Table VII. Our encryption method ensures data security without significantly increasing the computational burden and maintains high efficiency, even in environments with constrained resources. The method exhibits the scalability across datasets of varying sizes and different model architectures.

Computational and communication complexity: Let d denote the gradient's dimension for each client. In every training epoch, SMTFL performs: (i) random gradient splitting; (ii) intra-group local aggregation; (iii) score updating for poisoning detection; and (iv) key-share generation for threshold encryption. The computational complexity of each component is:

- Gradient splitting involves randomly generating g_1 and computing $g_2 = g - g_1$, incurring a complexity of $O(d)$.
- Intra-group local aggregation involves exchanging and linearly combining gradient shares among three clients, which incurs a complexity of $O(d)$.
- Score update consists of constant-time scalar operations, resulting in a complexity of $O(1)$.
- Key-share generation performs a polynomial evaluation of degree $(t-1)$ (with $t \ll d$), leading to a complexity of $O(t)$.

Thus, the per-epoch computational complexity per client is:

$$O(d) + O(d) + O(1) + O(t) \to O(d) \tag{28}$$

Each client transmits two masked gradient shares to group peers and one encrypted key share to the server. Thus, the communication complexity per client per epoch is $O(d)$.

Comparative complexity discussion: We find that SMTFL achieves privacy-preserving secure aggregation while matching the baseline FedAvg in both computational and communication complexity; in particular, it incurs $O(d)$ complexity without introducing any additional cryptographic overhead or accuracy loss. Existing privacy-preserving FL methods can be grouped into three main categories, whose per-epoch computation and communication complexity—together with those of FedAvg and SMTFL—are summarized in Table VI.

- HE-based methods rely on expensive encryption operations (e.g., homomorphic additions, relinearization, and modulus switching), resulting in computational complexity $O(\text{poly}(d,\lambda))$ and communication complexity $O(d \cdot \text{poly}(\lambda))$. Both may grow super-linearly in the dimension d and depend on its security parameter λ.
- MPC-based methods use multi-epoch secret sharing among k parties; while keeping the per-epoch computation on the order of $O(d)$, it introduces an $O(kd)$ communication complexity due to interactive k-party protocols. EIFFeL [13] is even more communication-intensive: it requires every pair of n clients to interact in each epoch, leading to $O(n^2)$-level scaling in both computation and communication, or $O(n^2 d)$ when expressed in terms of the gradient dimension.
- DP-based methods preserve the $O(d)$ level in both computation and communication, but necessarily inject noise into each update, causing non-negligible accuracy degradation, especially under non-IID data distributions.

Storage overhead: We compare the storage requirements per client. Specifically, prior to detecting malicious clients, the server retains historical gradients from the training process of the global model, such as each client's gradient updates for every epoch. Utilizing this stored historical gradients, the server estimates the gradient of malicious clients for each epoch when removing the poisoning gradient from the global model via the FUL. In contrast to the method described in [43], which demands an average of 2 GB of additional storage per client for training on MNIST and FMNIST datasets, SMTFL requires less than 0.69 GB per client. Meanwhile, we conduct experiments on more datasets, including the complex CIFAR-10 dataset, which requires 0.25 GB of storage space per client.

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
COMPARISON SMTFL WITH BASELINE DEFENSES WORKS

Dataset	No attack	Attack method	Fedavg	Krum	Median	Clipping Median	Trimmed Mean	Cos defense	DDFed	**Ours**
MNIST	98.57	IPM	90.52	21.07	94.35	74.22	84.80	96.93	95.56	**96.30**
		Scaling	10.27	50.40	95.92	94.16	9.79	84.71	90.51	**96.08**
		ALIE	97.28	37.22	95.55	93.62	96.45	96.23	96.60	**97.15**
FMNIST	86.19	IPM	71.00	27.11	75.52	71.86	61.97	79.97	79.29	**85.39**
		Scaling	9.99	51.62	77.82	74.93	71.00	69.82	73.45	**85.10**
		ALIE	80.82	46.54	76.49	74.00	79.13	79.12	79.26	**85.26**
CIFAR-10	67.78	IPM	37.84	25.31	39.38	23.45	40.79	52.44	43.73	**65.78**
		Scaling	37.20	25.29	48.86	22.51	40.32	54.89	55.04	**64.11**
		ALIE	45.92	37.64	40.17	28.69	65.89	55.74	59.15	**66.52**

D. Comparison of Baseline Defense Works

Section II presents a qualitative comparison of SMTFL and related works (see Table I). Here, we conduct a quantitative comparison, evaluating SMTFL against the mentioned baseline defenses. To better demonstrate the SMTFL's advantages, we adopt the same experimental settings as the compared works, including datasets, the number of clients, data distribution, and attack methods. Specifically, we use three datasets—MNIST, FMNIST, and CIFAR-10, $rate_{iid}$ is set to 0.2 and the number of clients is set to 100. For attack methods, we adopt the Inner Product Manipulation (IPM) attack, the Scaling attack, and the "a little Is enough" (ALIE) attack from DDFed [18]. As shown in Table VIII, The results show that, in the presence of attacks, SMTFL achieves the optimal defense capability compared to existing defense strategies, as the accuracy degradation of the global model relative to the non-attacked scenario is minimized. In particular, in terms of locating malicious clients, although the recently proposed DDFed and SMTFL can both identify malicious clients with an acc_{loc} exceeding 95% , SMTFL maintains a false positive rate for honest clients within 5% , which is 6% lower than that of DDFed.

E. System-Level Communication and Robustness Analysis

Bandwidth consumption: Each client transmits two masked gradient shares to peers and receives two in return, resulting in per-epoch uplink and downlink bandwidth costs of $O(d)$, matching FedAvg. Only one client uploads the aggregated share to the server, so the amortized uplink cost per client remains unchanged. The additional upload of encrypted key shares is constant and independent of d.

Latency analysis: The intra-group peer-to-peer exchanges introduce a small constant delay before global aggregation. Since all three clients in a group communicate in parallel, the latency increase is bounded by the time required for a single peer-to-peer transfer. This additional delay arises from the extra transmission overhead caused by gradient splitting, and its impact on the overall epoch time is minimal, depending on the network conditions and transmission rates.

Client dropouts and stragglers: It is important to note that in this work, we assume all clients participate 100% in each training epoch without dropout. We now consider the impact of occasional client dropouts or exits under normal conditions. SMTFL is designed to tolerate temporary dropouts. If one or two members of a group fail to respond in a given epoch, the affected group skips the local mixing step while all other groups proceed normally. Dynamic regrouping occurs in the subsequent epoch to restore full groups. As a result, only a small fraction of updates are temporarily delayed, and the global model continues to converge.

Groups with fewer than three clients: SMTFL relies on three-client mixing to ensure privacy; thus, any group with fewer than three clients temporarily suspends its update for that epoch. However, with 150 participating clients and randomized grouping in each epoch, the expected number of clients excluded from training is less than $\frac{1}{50}$ per epoch, resulting in a negligible impact on overall performance. Furthermore, regrouping in subsequent epochs allows these clients to promptly resume participation.

F. Applicability and Limitations of Linear Rollback

It is important to note that our unlearning mechanism, based on subtracting accumulated gradients, serves as an efficient approximation for removing a client's influence.

Non-Convexity: For non-convex models (e.g., deep neural networks), exact unlearning theoretically requires retraining from scratch. However, SMTFL leverages the property of local linearity: assuming small learning rates and frequent aggregations, the subtraction of gradient residuals provides a sufficiently accurate approximation to restore the model to a state as if the target client had not participated.

Adaptive Optimizers: The current formulation is derived primarily for SGD-based optimizers. We acknowledge that for adaptive optimizers (e.g., Adam), which maintain internal states (e.g., momentum and variance estimates) dependent on historical gradients, simple linear rollback may not fully eliminate the influence of the removed data. Adapting the rollback mechanism to account for these internal optimizer states remains a challenge for future research.

VIII. DISCUSSION AND CONCLUSION

In FL system, this paper introduces the SMTFL approach. For privacy preservation, SMTFL avoids injecting noise—which may degrade model performance—to obscure gradients. Instead, it leverages a checks-and-balances among client groups to prevent gradient inference attacks and resist collusion, ensuring that no participant can access others' gradients. For defending against poisoning attacks, SMTFL does not rely on trusted hardware, trusted clients, or public training data. Rather than depending on majority-voted gradient directions, which can be manipulated through collusion, SMTFL evaluates the contribution of each gradient to the global model based on performance

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.

changes. This enables reliable discrimination between benign and malicious updates. The effectiveness of SMTFL has been validated through extensive image classification experiments.

In practical FL deployments, SMTFL introduces only a single additional group-level communication process, without requiring extra interaction epochs between clients and the server, making it suitable for bandwidth-constrained systems. Its group-based structure naturally accommodates heterogeneous devices—clients with different computational capabilities can participate without relying on trusted hardware or stable high-speed connections. These properties enable SMTFL to be effectively deployed in real mobile computing and edge computing environments, where communication constraints, device heterogeneity, and potential untrustworthiness are prevalent.

In the future, we will attempt the following work:

- We will measure to reduce the impact on mistakenly targeted clients, such as optimizing group partitioning strategies and unified evaluation strategies.
- Without disclosing the gradients, we will mitigate the impact of poisoning gradients on the global model by exploring two ideas: 1) Preventing the poisoning gradients from being aggregated into the global model. 2) Although poisoning gradients have been aggregated into the global model, simplifying the gradients' storage, encryption, and decryption processes on the storage server, and efficiently decrypting and utilizing FUL to reduce the impact of poisoning gradients on the global model.
- Model training methods that support multimodal data are of great importance. We believe that the grouping strategy of SMTFL is applicable to training with different types of data. We will adjust SMTFL to accommodate more complex real-world scenarios.
- We will design and integrate dynamic secret sharing mechanisms to enable seamless key management and participant updates in open FL scenarios, thereby eliminating the need for complete system re-initialization when clients join or leave.

As the demands for large-scale model training grow across domains such as large language models, embodied AI, and edge intelligence, FL increasingly relies on diverse and resource-constrained devices. Ensuring the correctness of global models and protecting client privacy is more critical than ever. By avoiding assumptions about trusted hardware and supporting operation under heterogeneous and bandwidth-limited settings, SMTFL strengthens the practical deploy ability of federated learning and enhances reliability in real-world mobile and edge computing environments.

ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers for their insightful comments and suggestions.

REFERENCES

[1] C. Zhang et al., "A survey on federated learning," *Knowl.-Based Syst.*, vol. 216, 2021, Art. no. 106775.

[2] X. Kang et al., "Semi-asynchronous online federated crowdsourcing," in *Proc. IEEE 40th Int. Conf. Data Eng.*, 2024, pp. 4180–4193.

[3] J. Xu et al., "Agic: Approximate gradient inversion attack on federated learning," in *Proc. 41st Int. Symp. Reliable Distrib. Syst.*, 2022, pp. 12–22.

[4] Y. Wan, Y. Qu, W. Ni, Y. Xiang, L. Gao, and E. Hossain, "Data and model poisoning backdoor attacks on wireless federated learning, and the defense mechanisms: A comprehensive survey," *IEEE Commun. Surv. Tuts.*, vol. 26, no. 3, pp. 1861–1897, Third Quarter, 2024.

[5] H. Fereidooni et al., "SAFELearn: Secure aggregation for private federated learning," in *Proc. IEEE Secur. Privacy Workshops*, 2021, pp. 56–62.

[6] A. Li et al., "Efficient and privacy-preserving feature importance-based vertical federated learning," *IEEE Trans. Mobile Comput.*, vol. 23, no. 6, pp. 7238–7255, Jun. 2024.

[7] S. Truex et al., "LDP-Fed: Federated learning with local differential privacy," in *Proc. 3rd ACM Int. Workshop Edge Syst., Analytics Netw.*, 2020, pp. 61–66.

[8] J. Kim and S. Park, "Random gradient masking as a defensive measure to deep leakage in federated learning," 2024, *arXiv:2408.08430*.

[9] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," 2020, *arXiv:2012.13995*.

[10] M. S. Ozdayi, M. Kantarcioglu, and Y. R. Gel, "Defending against backdoors in federated learning with robust learning rate," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 9268–9276.

[11] X. Lyu et al., "Poisoning with cerberus: Stealthy and colluded backdoor attack against federated learning," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 9020–9028.

[12] H. Yang, W. Xi, Y. Shen, C. Wu, and J. Zhao, "RoseAgg: Robust defense against targeted collusion attacks in federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 2951–2966, 2024.

[13] A. Roy Chowdhury et al., "EIFFeL: Ensuring integrity for federated learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2535–2549.

[14] Z. Liu et al., "A survey on federated unlearning: Challenges, methods, and future directions," *ACM Comput. Surv.*, vol. 57, no. 1, pp. 1–38, 2024.

[15] H. Li, P. Rieger, S. Zeitouni, S. Picek, and A.-R. Sadeghi, "FLAIRS: FPGA-accelerated inference-resistant & secure federated learning," in *Proc. 33rd Int. Conf. Field- Program. Log. Appl.*, 2023, pp. 271–276.

[16] M. Naseri, J. Hayes, and E. D. Cristofaro, "Toward robustness and privacy in federated learning: Experimenting with local and central differential privacy," 2020, *arXiv:2009.03561*.

[17] H. Fereidooni et al., "FreqFed: A frequency analysis-based approach for mitigating poisoning attacks in federated learning," 2023, *arXiv:2312.04432*.

[18] R. Xu et al., "Dual defense: Enhancing privacy and mitigating poisoning attacks in federated learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 7076–70498.

[19] C. Fung, C. J. Yoon, and I. Beschastnikh, "The limitations of federated learning in sybil settings," in *Proc. 23rd Int. Symp. Res. Attacks, Intrusions Defenses*, 2020, pp. 301–316.

[20] S. Andreina, G. A. Marson, H. Möllering, and G. Karame, "BaFFLe: Backdoor detection via feedback-based federated learning," in *Proc. IEEE Int. Conf. Distrib. Comput. Syst.*, 2021, pp. 852–863.

[21] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private language models without losing accuracy," Oct. 18, 2017. [Online]. Available: http://arxiv.org/abs/1710.06963

[22] T. D. Nguyen et al., "{FLAME}: Taming backdoors in federated learning," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 1415–1432.

[23] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 118–128.

[24] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," 2021, *arXiv:1803.01498*. [Online]. Available: https://arxiv.org/abs/1803.01498

[25] D. Pasquini, D. Francati, and G. Ateniese, "Eluding secure aggregation in federated learning via model inconsistency," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2429–2443.

[26] Z. Zhao, H. Bin, H. Li, N. Yu, H. Zhu, and L. Sun, "FeaShare: Feature sharing for computation correctness in edge preprocessing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 11029–11044, Dec. 2024.

[27] Z. Zhao, Y. Zeng, J. Wang, H. Li, H. Zhu, and L. Sun, "Detection and incentive: A tampering detection mechanism for object detection in edge computing," in *Proc. 41st Int. Symp. Reliable Distrib. Syst.*, 2022, pp. 166–177.

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.

[28] X. Zhu, G. Li, and W. Hu, "Heterogeneous federated knowledge graph embedding learning and unlearning," in *Proc. ACM Web Conf.*, 2023, pp. 2444–2454.
[29] X. Gao et al., "VeriFi: Towards verifiable federated unlearning," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 6, pp. 5720–5736, Nov./Dec. 2024.
[30] K. Yu et al., "A blockchain-based shamir's threshold cryptography scheme for data protection in industrial Internet of Things settings," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 8154–8167, Jun. 2022.
[31] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
[32] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.
[33] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik, "EMNIST: Extending MNIST to handwritten letters," in *Proc. Int. Joint Conf. Neural Netw.*, 2017, pp. 2921–2926.
[34] A. Krizhevsky et al., "Learning multiple layers of features from tiny images," Tech. Rep. Univ. Toronto, 2009.
[35] R. Yang, Y. Deng, A. Zhu, X. Tong, and Z. Chen, "Few shot learning based on the street view house numbers (SVHN) dataset," in *Proc. Int. Conf. Edge Comput. IoT*, Springer, 2020, pp. 86–102.
[36] M. Fang, X. Cao, J. Jia, and N. Gong, "Local model poisoning attacks to {Byzantine-Robust} federated learning," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 1605–1622.
[37] P. Rieger, T. D. Nguyen, M. Miettinen, and A.-R. Sadeghi, "Deepsight: Mitigating backdoor attacks in federated learning through deep model inspection," 2022, *arXiv:2201.00763*.
[38] S. Shen, S. Tople, and P. Saxena, "Auror: Defending against poisoning attacks in collaborative deep learning systems," in *Proc. Annu. Conf. Comput. Secur. Appl.*, 2016, pp. 508–519.
[39] A. Madry et al., "Towards deep learning models resistant to adversarial attacks," *Stat*, vol. 1050, no. 9, 2017.
[40] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," 2019, *arXiv:1906.08935*. [Online]. Available: https://arxiv.org/abs/1906.08935
[41] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of fedavg on non-IID data," 2019, *arXiv:1907.02189*.
[42] D. N. Yaldiz, T. Zhang, and S. Avestimehr, "Secure federated learning against model poisoning attacks via client filtering," 2023, *arXiv:2304.00160*.
[43] X. Cao, J. Jia, Z. Zhang, and N. Z. Gong, "Fedrecover: Recovering from poisoning attacks in federated learning using historical information," in *Proc. IEEE Symp. Secur. Privacy*, 2023, pp. 1366–1383.



Zhihui Zhao (Member, IEEE) received the BS degree from the Dalian University of Technology, China, in 2018, and the PhD degree from the Institute of Information Engineering, Chinese Academy of Sciences, China, in 2024. He is a lecturer with the School of Computer Science and Technology, Taiyuan University of Technology. His primary research interests include edge computing security and IoT security.



Xiaorong Dong received the BS degree from the Taiyuan University of Technology, China, in 2023. She is currently working toward the MS degree with the Taiyuan University of Technology, China. Her primary research interests include federated learning security and IoT security.



Yimo Ren received the BS and MS degrees from Tianjin University, China, in 2018, and the PhD degree from the Institute of Information Engineering, Chinese Academy of Sciences, China, in 2024. He is a associate researcher with the Institute of Information Engineering. His research interests include network security and artificial intelligence.



Jianhua Wang (Member, IEEE) received the BS and MS degrees from the Taiyuan University of Technology, China, in 2017 and 2020, respectively, and the PhD degree from Beijing Jiaotong University, China, in 2024. He is a lecturer with the College of Computer Science and Technology, Taiyuan University of Technology. His research interest is federated learning security.



Dan Yu (Member, IEEE) received the BS degree from the North University of China, in 2007, the MS degree from the Beijing University of Posts and Telecommunications, in 2013, and the PhD degree from the Taiyuan University of Technology, China, in 2020. She is a lecturer with the College of Computer Science and Technology, Taiyuan University of Technology. Her research interest includes IoT security.



Hongsong Zhu (Member, IEEE) received the BE degree from the University of Science and Technology of China, in 1995, the ME degree from the Institute of Software, Chinese Academy of Sciences, in 2000, and the PhD degree from the Institute of Computing Technology, Chinese Academy of Sciences, in 2009. He is the professor with the Institute of Information Engineering, CAS. His recent research has been focused on IoT security, network attack and defend, and security Big Data analysis.



Yongle Chen (Member, IEEE) received the BS degree from Jilin University, in 2007, the MS degree from the Institute of Software, Chinese Academy of Science, in 2009, and the PhD degree from the University of Chinese Academy of Sciences, China, in 2013. He is currently a professor with the College of Artificial Intelligence, Taiyuan University of Technology, China. His research interests include cyber-physical systems and IoT security.

Authorized licensed use limited to: Anhui University. Downloaded on June 29,2026 at 08:39:46 UTC from IEEE Xplore. Restrictions apply.